GST Invoice Manager - Complete Project Documentation

Table of Contents

1. Project Overview
2. Why This Project Exists
3. What Problem It Solves
4. Technology Stack
5. Project Architecture
6. Directory Structure
7. Data Models & Types
8. Core Business Logic
9. Component Deep-Dive
10. Data Flow & State Management
11. Feature Walkthrough
12. CSV Import/Export System
13. Validation Rules
14. Indian GST Explained (For Non-Tax People)
15. How to Run the Project
16. UI Component Library
17. Known Limitations
18. Possible Future Enhancements

---

1. Project Overview

GST Invoice Manager is a fully client-side web application built with Next.js for managing Indian business invoices with complete Goods and Services Tax (GST) calculations. It allows users to create, edit, cancel, delete, search, sort, import (CSV), and export (CSV) both **Sale** and **Purchase** invoices, with a detailed GST breakup across all standard Indian tax rate slabs.

Key facts:

- **No backend/server/database** - runs entirely in the browser
- **Data persists in browser localStorage** - survives page refreshes and browser restarts
- **Handles all 5 Indian GST rate slabs**: 28%, 18%, 12%, 5%, 3%
- **Supports both CGST+SGST (intra-state) and IGST (inter-state)** tax types
- **Supports both tax-inclusive and tax-exclusive** calculation modes
- **Full CSV round-trip** - export invoices to CSV, import them back

---

2. Why This Project Exists

Indian businesses are required by law to maintain detailed records of all sale and purchase invoices with proper GST breakup. Many small businesses and accountants manage these records in Excel spreadsheets, which is error-prone and tedious.

This application provides:

- **Automated GST calculations** - no manual math errors
- **Structured data entry** - enforced formats for invoice numbers, dates, and GSTIN
- **One-click export to CSV** - compatible with Excel/Google Sheets for further analysis or filing
- **Bulk import from CSV** - migrate existing data from spreadsheets
- **Instant summaries** - see total basic amount, total tax, and total amount at a glance

---

3. What Problem It Solves

Pain Point	How This App Solves It
Manual GST calculation errors	Automatic calculation engine handles inclusive/exclusive tax math
Tracking 15 different GST columns per invoice	Auto-distributes tax to correct CGST/SGST/IGST buckets
Maintaining separate sale vs purchase registers	Built-in tab system with separate data stores
Migrating from Excel to a proper tool	CSV import with validation and preview
Getting data out for tax filing	CSV export with all 15 GST columns + dates in DD-MM-YYYY format
Invoice number formatting	Auto-prefixes fiscal year (e.g., "2025-2026/INV001")
Duplicate invoice detection	Validates uniqueness before saving
Cancelled invoice handling	Proper cancellation (zeroes amounts, marks as cancelled, preserves record)

---

4. Technology Stack

Core Framework

Technology	Version	Purpose
Next.js	15.3.0	React-based full-stack framework (used as SPA here)
React	19	UI component library
TypeScript	5	Type-safe JavaScript
Tailwind CSS	3.4.17	Utility-first CSS framework

UI Component Library

Technology	Purpose
Radix UI (40+ packages)	Accessible, unstyled UI primitives (dialogs, selects, tabs, etc.)
Shadcn/ui	Pre-built Tailwind-styled components built on Radix
Lucide React	Icon library (Receipt, ShoppingCart, Plus, Trash2, etc.)
Class Variance Authority	CSS variant management for component styling

Forms & Validation

Technology	Version	Purpose
React Hook Form	7.54.1	Performant form handling
Zod	3.24.1	Schema-based validation (available, primarily custom validation used)
@hookform/resolvers	3.9.1	Bridges Zod schemas with React Hook Form

Utilities

Technology	Version	Purpose
date-fns	4.1.0	Date manipulation and formatting
Recharts	2.15.0	Chart visualization (available for dashboard)
Sonner	1.7.1	Toast notification library
next-themes	0.4.4	Dark/light mode theme management

Development Tools

Technology	Version	Purpose
PostCSS	8.5	CSS processing pipeline
ESLint	-	Code linting
tailwindcss-animate	1.0.7	Animation utilities for Tailwind

---

5. Project Architecture

High-Level Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────┐
│                              BROWSER                                 │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │                        Next.js App (SPA)                     │   │
│  │                                                              │   │
│  │  ┌──────────────────────────────────────────────────┐        │   │
│  │  │                  app/page.tsx                    │        │   │
│  │  │             (Main State Manager)                 │        │   │
│  │  │                                                  │        │   │
│  │  │  State: saleInvoices[], purchaseInvoices[]       │        │   │
│  │  │  State: showForm, showCSVImport, activeTab       │        │   │
│  │  │  State: editingInvoice, currentInvoiceType       │        │   │
│  │  └──────────────────────────────────────────────────┘        │   │
│  │            │            │            │                        │   │
│  │            │            │            │                        │   │
│  │    ┌───────▼────┐  ┌────▼───────┐  ┌▼──────────┐             │   │
│  │    │ Invoice    │  │ Invoice    │  │ CSV Import │             │   │
│  │    │ Form       │  │ Table      │  │ Frontend   │             │   │
│  │    │            │  │            │  │            │             │   │
│  │    │ - Create   │  │ - Search   │  │ - Upload   │             │   │
│  │    │ - Edit     │  │ - Sort     │  │ - Validate │             │   │
│  │    │ - Preview  │  │ - Cancel   │  │ - Preview  │             │   │
│  │    │ - Validate │  │ - Delete   │  │ - Import   │             │   │
│  │    └────────────┘  └────────────┘  └────────────┘             │   │
│  │         │              │               │                      │   │
│  │         │              │               │                      │   │
│  │    ┌────▼──────────────▼───────────────▼──────────────┐       │   │
│  │    │                lib/ (Business Logic)              │       │   │
│  │    │                                                  │       │   │
│  │    │  tax-calculations.ts  →  GST math engine         │       │   │
│  │    │  local-storage.ts     →  Data persistence        │       │   │
│  │    │  csv-parser.ts        →  CSV parse/format        │       │   │
│  │    │  types.ts             →  TypeScript interfaces   │       │   │
│  │    └──────────────────────────────────────────────────┘       │   │
│  │                              │                                 │   │
│  │                              │                                 │   │
│  │    ┌─────────────────────────▼────────────────────────┐       │   │
│  │    │              Browser localStorage                 │       │   │
│  │    │                                                   │       │   │
│  │    │  Key: "gst-sale-invoices"     → Invoice[]         │       │   │
│  │    │  Key: "gst-purchase-invoices" → Invoice[]         │       │   │
│  │    └───────────────────────────────────────────────────┘       │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  components/ui/  (40+ Shadcn/Radix UI components)            │   │
│  │  Card, Button, Input, Table, Select, Switch, Badge,          │   │
│  │  Dialog, Toast, Label, Tabs, etc.                            │   │
│  └──────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────┘
```

Architecture Decisions

1. **No Backend/Database**: Originally used Prisma (ORM) for database access, but was intentionally removed (commit `5e9d024`). The app is now 100% client-side with localStorage, making it:
    - Zero-cost to host (static site on Vercel/Netlify)
    - Zero-dependency on servers
    - Instantly deployable
    - Privacy-first (data never leaves the user's browser)
2. **Single-Page Application**: Although built with Next.js, it uses a single route (`app/page.tsx`) with conditional rendering to switch between views (form, table, CSV import).
3. **Component-Based**: Each major feature is encapsulated in its own component with props-based communication through the parent `page.tsx`.

---

6. Directory Structure

```
gst-invoice-manager/
│
├── app/                          # Next.js App Router
│   ├── layout.tsx                # Root HTML layout (meta tags, body wrapper)
│   ├── page.tsx                  # Main application component (state hub)
│   └── globals.css               # Global Tailwind styles + CSS variables
│
├── components/                   # Application components
│   ├── invoice-form.tsx          # Create/edit invoice form with tax items
│   ├── invoice-table.tsx         # Sortable, searchable invoice table
│   ├── invoice-type-selector.tsx # Sale vs Purchase type picker modal
│   ├── csv-import-frontend.tsx   # CSV file upload, parse, preview, import
│   ├── frontend-export.tsx       # CSV export (all/filtered/separate files)
│   ├── theme-provider.tsx        # Dark/light mode wrapper
│   └── ui/                       # ~40 Shadcn/Radix UI components
│       ├── button.tsx
│       ├── card.tsx
│       ├── input.tsx
│       ├── table.tsx
│       ├── select.tsx
│       ├── switch.tsx
│       ├── badge.tsx
│       ├── label.tsx
│       ├── dialog.tsx
│       ├── toast.tsx
│       ├── toaster.tsx
│       └── ... (accordion, alert-dialog, avatar, calendar,
│                carousel, checkbox, collapsible, command,
│                context-menu, dropdown-menu, drawer,
│                hover-card, menubar, navigation-menu,
│                popover, progress, radio-group, resizable,
│                scroll-area, separator, skeleton, slider,
│                tabs, textarea, toggle, toggle-group, tooltip)
│
├── hooks/                        # Custom React hooks
│   ├── use-toast.ts              # Toast notification hook (state + dispatch)
│   └── use-mobile.tsx            # Mobile viewport detection hook
│
├── lib/                          # Core business logic
│   ├── types.ts                  # TypeScript interfaces (Invoice, TaxItem, etc.)
│   ├── tax-calculations.ts       # GST calculation engine
│   ├── local-storage.ts          # localStorage CRUD service
│   ├── csv-parser.ts             # CSV parsing and date utilities
│   └── utils.ts                  # General utilities (cn() for class merging)
│
├── public/                       # Static assets (served as-is)
├── styles/                       # Additional style files
│
├── package.json                  # Dependencies and scripts
├── tsconfig.json                 # TypeScript configuration
├── tailwind.config.ts            # Tailwind CSS configuration
├── next.config.mjs               # Next.js configuration
```

```
├── postcss.config.mjs            # PostCSS configuration
└── components.json               # Shadcn UI configuration
```

---

7. Data Models & Types

All types are defined in `lib/types.ts`.

InvoiceType

```
type InvoiceType = "sale" | "purchase"
```

Two categories of invoices that are stored and displayed separately.

TaxType

```
type TaxType = "CGST_SGST" | "IGST"
```

- **CGST_SGST**: Used for **intra-state** transactions (buyer and seller in the same Indian state). The total tax is split 50/50 between Central GST and State GST.
- **IGST**: Used for **inter-state** transactions (buyer and seller in different states). The full tax goes to Integrated GST.

TaxItem

```
interface TaxItem {
  id: string          // Unique identifier for this line item
  amount: number      // The monetary amount for this item
  taxType: TaxType    // CGST_SGST or IGST
  taxRate: 28 | 18 | 12 | 5 | 3   // GST rate percentage
  inclusive: boolean   // true = amount includes tax, false = amount excludes tax
}
```

Each invoice can have **multiple** TaxItems with different rates and types. This allows a single invoice to have items at 18% CGST+SGST and other items at 5% IGST, for example.

GSTBreakup

```
interface GSTBreakup {
  // CGST+SGST pairs (rate shown is half of total rate)
  cgst_14: number    // CGST at 14% (half of 28%)
  sgst_14: number    // SGST at 14% (half of 28%)
  cgst_9: number     // CGST at 9%  (half of 18%)
  sgst_9: number     // SGST at 9%  (half of 18%)
  cgst_6: number     // CGST at 6%  (half of 12%)
  sgst_6: number     // SGST at 6%  (half of 12%)
  cgst_2_5: number   // CGST at 2.5% (half of 5%)
  sgst_2_5: number   // SGST at 2.5% (half of 5%)
  cgst_1_5: number   // CGST at 1.5% (half of 3%)
  sgst_1_5: number   // SGST at 1.5% (half of 3%)

  // IGST columns (full rate)
  igst_28: number    // IGST at 28%
  igst_18: number    // IGST at 18%
  igst_12: number    // IGST at 12%
  igst_5: number     // IGST at 5%
  igst_3: number     // IGST at 3%
}
```

This represents the **15 distinct tax columns** required for proper Indian GST record-keeping.

Invoice (Main Entity)

```
interface Invoice {
  id: string                // UUID, generated via crypto.randomUUID()
  type: InvoiceType         // "sale" or "purchase"
  date: string              // ISO date string (YYYY-MM-DD)
  invoiceNo: string         // Format: "YYYY-YYYY/PREFIX" (e.g., "2025-2026/INV001")
  party: string             // Name of the buyer/seller (stored uppercase)
  gstNumber?: string        // Optional 15-character GSTIN
  items: TaxItem[]          // Array of line items with tax details
  basicAmount: number       // Sum of all pre-tax amounts
  totalAmount: number       // Sum of all post-tax amounts
  gstBreakup: GSTBreakup    // Detailed tax distribution across 15 columns
  isCancelled?: boolean     // Whether the invoice has been cancelled
  cancelledAt?: string      // ISO timestamp of cancellation
  cancelledReason?: string  // Reason for cancellation
  createdAt?: string        // ISO timestamp of creation
  updatedAt?: string        // ISO timestamp of last update
}
```

Entity Relationship

```
Invoice (1) ──────► (many) TaxItem
    │
    └──────► (1) GSTBreakup
```

Each Invoice contains:

- One or more TaxItems (the individual amounts with their tax types and rates)
- Exactly one GSTBreakup (the aggregated result of all TaxItem calculations)

---

8. Core Business Logic

8.1 Tax Calculation Engine (`lib/tax-calculations.ts`)

The heart of the application. The `calculateTax(items: TaxItem[])` function processes an array of TaxItems and returns:

- `basicAmount` - total pre-tax amount
- `totalAmount` - total post-tax amount
- `gstBreakup` - detailed breakup across all 15 GST columns

Calculation Modes

Mode 1: Tax Exclusive (`inclusive: false`)

```
Input: amount = 1000, taxRate = 18%
──────────────────────────────────────
Basic Amount = 1000.00        (amount as-is)
Tax Amount   = 180.00         (1000 × 18/100)
Total        = 1180.00        (basic + tax)
```

Mode 2: Tax Inclusive (`inclusive: true`)

```
Input: amount = 1180, taxRate = 18%
──────────────────────────────────────
Tax Multiplier = 1.18         (1 + 18/100)
Basic Amount   = 1000.00      (1180 / 1.18)
Tax Amount     = 180.00       (1180 - 1000)
Total          = 1180.00      (the original input)
```

Tax Distribution Logic

After computing the tax amount for each item:

If `taxType = "CGST_SGST"` :

```
halfTax = taxAmount / 2

taxRate 28% → cgst_14 += halfTax, sgst_14 += halfTax
taxRate 18% → cgst_9  += halfTax, sgst_9  += halfTax
taxRate 12% → cgst_6  += halfTax, sgst_6  += halfTax
taxRate  5% → cgst_2_5 += halfTax, sgst_2_5 += halfTax
taxRate  3% → cgst_1_5 += halfTax, sgst_1_5 += halfTax
```

If `taxType = "IGST"`:

```
taxRate 28% → igst_28 += taxAmount
taxRate 18% → igst_18 += taxAmount
taxRate 12% → igst_12 += taxAmount
taxRate  5% → igst_5  += taxAmount
taxRate  3% → igst_3  += taxAmount
```

Complete Example

```
Invoice with 2 items:
  Item 1: ₹10,000 at 18% CGST+SGST (exclusive)
  Item 2: ₹5,000 at 12% IGST (exclusive)

Item 1 Calculation:
  Basic = 10,000
  Tax = 10,000 × 18/100 = 1,800
  Half tax = 900
  → cgst_9 = 900, sgst_9 = 900

Item 2 Calculation:
  Basic = 5,000
  Tax = 5,000 × 12/100 = 600
  → igst_12 = 600

Invoice Totals:
  Basic Amount = 10,000 + 5,000 = 15,000
  Total Amount = 11,800 + 5,600 = 17,400

GST Breakup:
  cgst_9 = 900, sgst_9 = 900
  igst_12 = 600
  (all other columns = 0)
```

8.2 Data Persistence (`lib/local-storage.ts`)

The `LocalStorageService` class manages all data persistence through browser's localStorage API.

Storage Keys

```
"gst-sale-invoices"     → JSON array of sale Invoice objects
"gst-purchase-invoices" → JSON array of purchase Invoice objects
```

API

Method	Description
`getInvoices(type)`	Reads and parses invoices from localStorage for "sale" or "purchase"
`saveInvoices(type, invoices)`	Writes entire invoice array to localStorage
`addInvoice(invoice)`	Generates UUID, appends to existing invoices, saves
`addInvoices(newInvoices)`	Batch import with duplicate invoice number checking. Returns `{ successful[], failed[] }`
`exportToCSV(type?)`	Generates CSV string with 23 columns. Optional type filter.

CSV Export Format (23 columns)

```
TYPE, DATE, INVOICE NO., PARTY, GST NUMBER, STATUS, BASIC AMOUNT,
CGST (14%), SGST (14%), CGST (9%), SGST (9%), CGST (6%), SGST (6%),
CGST (2.5%), SGST (2.5%), CGST (1.5%), SGST (1.5%),
IGST (28%), IGST (18%), IGST (12%), IGST (5%), IGST (3%),
TOTAL AMOUNT
```

8.3 CSV Parser (`lib/csv-parser.ts`)

`parseCSV(csvText: string): CSVRow[]`

Custom CSV parser that handles:

- Quoted fields (with comma inside quotes)
- Escaped quotes (`""` inside quoted fields)
- Empty lines and whitespace

Date Utilities

- `formatDateToDDMMYYYY(dateString)` - Converts ISO date to DD-MM-YYYY
- `parseDateFromDDMMYYYY(dateString)` - Converts DD-MM-YYYY to ISO date

8.4 Fiscal Year Calculation

```
function getFiscalYear(dateString: string): string {
  const date = new Date(dateString)
  const month = date.getMonth() // 0-indexed
  const year = date.getFullYear()

  if (month >= 3) {           // April (3) onwards
    return `${year}-${year + 1}`    // e.g., "2025-2026"
  }
  return `${year - 1}-${year}`      // Jan-Mar: e.g., "2024-2025"
}
```

Indian fiscal year runs **April to March**:

- Date: 15-Jun-2025 → Fiscal Year: "2025-2026"
- Date: 10-Feb-2026 → Fiscal Year: "2025-2026"
- Date: 05-Apr-2026 → Fiscal Year: "2026-2027"

8.5 Smart Tax Reconstruction (for Editing)

When loading an existing invoice for editing, the original TaxItem array may not be stored. The app **reconstructs** TaxItems from the GSTBreakup using reverse math:

```
For each CGST+SGST pair where both values > 0:
  totalTax = cgst + sgst
  basicAmount = (totalTax × 100) / rate
  → Create TaxItem { amount: basicAmount, taxType: "CGST_SGST", rate, inclusive: false }

For each IGST column where value > 0:
  basicAmount = (igstAmount × 100) / rate
  → Create TaxItem { amount: basicAmount, taxType: "IGST", rate, inclusive: false }

Fallback (no tax items found but amounts exist):
  → Estimate rate from totalAmount/basicAmount ratio
  → Create single TaxItem with best-guess rate
```

---

9. Component Deep-Dive

9.1 Main Page (`app/page.tsx`)

Role: Central state manager and orchestrator for the entire application.

State Variables:

State	Type	Purpose

`saleInvoices`	`Invoice[]`	All sale invoices loaded from localStorage
`purchaseInvoices`	`Invoice[]`	All purchase invoices loaded from localStorage
`showTypeSelector`	`boolean`	Show the sale/purchase picker
`showForm`	`boolean`	Show the invoice creation/edit form
`showCSVImport`	`boolean`	Show the CSV import panel
`currentInvoiceType`	`InvoiceType | null`	Currently selected invoice type
`editingInvoice`	`Invoice | null`	Invoice being edited (null = creating new)
`activeTab`	`InvoiceType`	Currently active tab (sale or purchase)
`loading`	`boolean`	Loading state while reading localStorage

Key Handlers:

Handler	What It Does
`loadInvoices()`	Reads both sale + purchase invoices from localStorage
`handleAddNewInvoice()`	Opens the type selector modal
`handleInvoiceTypeSelect(type)`	Sets type, switches tab, opens form
`handleAddInvoice(invoice)`	Creates new or updates existing invoice in localStorage
`handleCSVImportComplete(results)`	Reloads data after CSV import, shows success/failure toast
`handleEditInvoice(invoice)`	Opens form with invoice data pre-filled
`handleDeleteInvoice(id)`	Removes invoice from localStorage and state
`handleCancelInvoice(id)`	Marks invoice cancelled, zeroes all amounts
`handleCancelForm()`	Resets all modal/form states

UI Layout (conditional rendering):

```
┌──────────────────────────────────────┐
│  Header Card                         │
│  [Add New Invoice] [Import CSV]      │
│  [Export All CSV] [Export Sep.]      │
├──────────────────────────────────────┤
│                                      │
│  IF showTypeSelector:                │
│    → InvoiceTypeSelector             │
│                                      │
│  IF showCSVImport:                   │
│    → CSVImportFrontend               │
│                                      │
│  IF showForm:                        │
│    → InvoiceForm                     │
│                                      │
│  ELSE (default view):                │
│    → Tab Buttons (Sale|Purchase)     │
│    → InvoiceTable                    │
│    → Summary Card                    │
│                                      │
└──────────────────────────────────────┘
```

9.2 Invoice Form (`components/invoice-form.tsx`)

Props:

Prop	Type	Purpose
`onSubmit`	`(invoice: Invoice) => void`	Callback when form is submitted
`onCancel`	`() => void`	Callback when form is cancelled
`editingInvoice`	`Invoice | null`	Pre-fill form for editing
`invoiceType`	`InvoiceType`	Sale or purchase
`existingInvoices`	`Invoice[]`	Used for duplicate invoice number checking
`submitting`	`boolean`	Disable form while saving

Internal State:

- `items: TaxItem[]` - Dynamic list of tax line items (starts with 1)
- `formData` - Date, invoice prefix, party name, GST number
- `calculatedData` - Live-calculated basic amount, total, and GST breakup

Key Behaviors:

1. **Live Calculation**: Recalculates whenever any tax item changes (via `useEffect`)
2. **Fiscal Year Auto-Prefix**: Invoice number shows "YYYY-YYYY/" prefix based on selected date
3. **Dynamic Items**: Users can add/remove multiple tax items with different rates
4. **Smart Edit Loading**: When editing, reconstructs TaxItems from GSTBreakup
5. **Validation on Submit**: Required fields, GST number format, duplicate check, at least one valid item

9.3 Invoice Table (`components/invoice-table.tsx`)

Props:

Prop	Type	Purpose
`invoices`	`Invoice[]`	List of invoices to display
`onEdit`	`(invoice: Invoice) => void`	Edit callback
`onDelete`	`(id: string) => void`	Delete callback
`onCancel`	`(id: string) => void`	Cancel callback
`invoiceType`	`InvoiceType`	Used in display text

Features:

- **Search**: Filters by invoice number, party name, or GST number (case-insensitive)
- **Sort**: Click column headers to sort by date, invoice no., party, basic amount, or total amount. Click again to reverse direction.
- **GST Column Toggle**: "Show GST" / "Hide GST" button to expand/collapse the 15 GST columns
- **Totals Row**: Bottom row shows sum of basic amount, tax, and total for all visible invoices
- **Status Badges**: Green "ACTIVE" or red "CANCELLED" badge per invoice
- **Actions**: Edit (disabled for cancelled), Cancel (confirmation dialog), Delete (confirmation dialog)
- **Cancelled Styling**: Cancelled rows have 60% opacity and red background tint

9.4 Invoice Type Selector (`components/invoice-type-selector.tsx`)

A simple modal with two large buttons:

- **Sale Invoice** (green icon) - "Invoice for goods/services sold"
- **Purchase Invoice** (blue icon) - "Invoice for goods/services purchased"

9.5 CSV Import (`components/csv-import-frontend.tsx`)

Three-step flow:

1. **File Selection**: Upload CSV file + download sample template
2. **Processing**: Parse CSV, validate each row, show preview table with status icons
3. **Import**: Import valid records to localStorage, skip invalid ones

Validation per row:

- Required: DATE, INVOICE NO., PARTY
- Date format: DD-MM-YYYY, DD/MM/YYYY, or YYYY-MM-DD
- Amounts must be non-negative
- Skips summary/header rows (detects "TOTAL", "MAY")
- Smart column matching (case-insensitive, partial match)

9.6 Frontend Export (`components/frontend-export.tsx`)

Two export modes:

1. **Single file**: All invoices (or filtered by current tab type)
2. **Separate files**: Creates two downloads - one for sale, one for purchase

File naming: `Sale_Invoices_DD-MM-YYYY.csv`, `Purchase_Invoices_DD-MM-YYYY.csv`

---

10. Data Flow & State Management

Creating an Invoice

```
User clicks "Add New Invoice"
    │
    ▼
InvoiceTypeSelector shown
    │
    ▼ (user picks "Sale" or "Purchase")
    │
InvoiceForm shown (with invoiceType prop)
    │
    ▼ (user fills date, invoice#, party, GST#, adds tax items)
    │
    │  ┌──────────────────────────────────┐
    │  │  useEffect runs on every item change │
    │  │  calculateTax(items) → sets          │
    │  │  calculatedData (basic, total, GST)  │
    │  └──────────────────────────────────┘
    │
    ▼ (user clicks "Save Invoice")
    │
handleSubmit validates:
    ├── Invoice prefix + party required
    ├── GST number format (if provided)
    ├── At least one item with amount > 0
    └── No duplicate invoice number
    │
    ▼ (all valid)
    │
Constructs Invoice object with:
    ├── Fiscal year prefix + user prefix → invoiceNo
    ├── Party name uppercased
    ├── calculatedData.basicAmount, totalAmount, gstBreakup
    │
    ▼
page.tsx handleAddInvoice(invoice) called
    │
    ├── localStorageService.addInvoice(invoice)
    │      └── Generates UUID, appends to localStorage array
    │
    ├── loadInvoices() → refreshes React state
    │
    └── Toast: "Invoice Added"
```

Cancelling an Invoice

```
User clicks Cancel (Ban icon) on a row
    │
    ▼
Confirmation dialog: "Are you sure?"
    │
    ▼ (user confirms)
    │
handleCancelInvoice(id):
    ├── Finds invoice by ID
    ├── Sets isCancelled = true
    ├── Sets cancelledAt = now
    ├── Sets ALL amounts to 0 (basicAmount, totalAmount, entire gstBreakup)
    ├── Saves to localStorage
    └── Updates React state
```

CSV Import Flow

```
User clicks "Import CSV"
    │
    ▼
CSVImportFrontend shown
    │
    ▼ (user uploads .csv file)
    │
handleProcessFile():
    ├── Reads file as text
    ├── parseCSV(text) → CSVRow[]
    ├── For each row: validateAndParseInvoice(row)
    │     ├── Extracts columns (case-insensitive matching)
    │     ├── Validates required fields
    │     ├── Parses date (supports 3 formats)
    │     ├── Parses amounts and GST breakup
    │     ├── Reconstructs TaxItems from GST columns
    │     └── Returns { data, errors, isValid }
    │
    ▼
Preview table shown with green/red status per row
    │
    ▼ (user clicks "Import N Valid Records")
    │
handleImport():
    ├── Filters valid invoices
    ├── localStorageService.addInvoices(validInvoices)
    │     └── Checks for duplicate invoice numbers
    │         ├── Duplicates → added to failed[]
    │         └── Unique → saved to localStorage
    │
    ▼
page.tsx handleCSVImportComplete(results):
    ├── loadInvoices() → refreshes state
    ├── Toast: success/failure counts
    └── Switches to appropriate tab
```

---

11. Feature Walkthrough

11.1 Dashboard View (Default)

When the app loads, you see:

1. **Header**: App title, description, action buttons (Add New, Import CSV, Export)
2. **Tab Bar**: "Sale Invoices (N)" and "Purchase Invoices (N)" buttons
3. **Invoice Table**: Searchable, sortable list of invoices for the active tab
4. **Summary Card**: Shows active/cancelled counts, basic amount total, tax total, total amount

11.2 Adding a Sale Invoice

1. Click "Add New Invoice" → Pick "Sale Invoice"
2. Select date (determines fiscal year prefix automatically)
3. Enter invoice number suffix (e.g., "INV001") → shown as "2025-2026/INV001"
4. Enter party name (auto-uppercased)
5. Optionally enter GSTIN (15-char, validated)
6. First tax item is pre-filled: ₹0 at 18% CGST+SGST (inclusive)
7. Enter amount, change tax type/rate/inclusive as needed
8. Click "Add Item" for more line items at different rates
9. Watch live calculation preview update in real-time
10. Click "Save Invoice" → validated → saved → redirected to table

11.3 Editing an Invoice

1. Click pencil icon on a table row (disabled for cancelled invoices)
2. Form opens pre-filled with existing data
3. Tax items are **reconstructed** from GST breakup (since the original items may not be perfectly stored)
4. Make changes → "Update Invoice" → duplicate check excludes self → saved

11.4 Cancelling an Invoice

1. Click ban icon on a table row
2. Confirmation dialog appears
3. On confirm: invoice amounts are zeroed out, marked as CANCELLED
4. Invoice remains visible in the table (greyed out) for audit trail
5. Cannot be edited after cancellation

11.5 Importing from CSV

1. Click "Import CSV"
2. Optionally download sample template to see expected format
3. Upload your CSV file
4. Click "Process CSV" → validation runs on every row
5. Preview table shows valid rows (green checkmark) and invalid rows (red X with error messages)
6. Toggle "Show All GST Columns" to verify GST breakup values
7. Click "Import N Valid Records" → only valid, non-duplicate records are imported
8. Toast shows results (e.g., "8 imported successfully, 2 failed")

11.6 Exporting to CSV

From header: "Export All CSV" exports both sale + purchase in one file **From header**: "Export Separate Files" creates two separate downloads **From tab bar**: "Export Sale" or "Export Purchase" exports only the active tab

Files include:

- All 23 columns (type, date, invoice no, party, GST number, status, basic amount, 15 GST columns, total amount)
- Dates in DD-MM-YYYY format

- Amounts with 2 decimal places

---

12. CSV Import/Export System

Export Column Specification (23 columns)

#	Column Name	Example	Notes
1	TYPE	SALE	"SALE" or "PURCHASE"
2	DATE	15-06-2025	DD-MM-YYYY format
3	INVOICE NO.	2025-2026/INV001	Full fiscal year prefix
4	PARTY	ABC TRADERS	Uppercase
5	GST NUMBER	22AAAAA0000A1Z5	15-char GSTIN or empty
6	STATUS	ACTIVE	"ACTIVE" or "CANCELLED"
7	BASIC AMOUNT	10000.00	Pre-tax sum
8	CGST (14%)	0.00	Tax at 14% (half of 28%)
9	SGST (14%)	0.00	Tax at 14% (half of 28%)
10	CGST (9%)	900.00	Tax at 9% (half of 18%)
11	SGST (9%)	900.00	Tax at 9% (half of 18%)
12	CGST (6%)	0.00	Tax at 6% (half of 12%)
13	SGST (6%)	0.00	Tax at 6% (half of 12%)
14	CGST (2.5%)	0.00	Tax at 2.5% (half of 5%)
15	SGST (2.5%)	0.00	Tax at 2.5% (half of 5%)
16	CGST (1.5%)	0.00	Tax at 1.5% (half of 3%)
17	SGST (1.5%)	0.00	Tax at 1.5% (half of 3%)
18	IGST (28%)	0.00	Inter-state at 28%
19	IGST (18%)	0.00	Inter-state at 18%
20	IGST (12%)	0.00	Inter-state at 12%
21	IGST (5%)	0.00	Inter-state at 5%
22	IGST (3%)	0.00	Inter-state at 3%
23	TOTAL AMOUNT	11800.00	Post-tax sum

Import Requirements

- File must be `.CSV` format
- First row must be headers (flexible matching - case insensitive, partial match)
- Required columns: DATE, INVOICE NO., PARTY

- Supported date formats: `DD-MM-YYYY`, `DD/MM/YYYY`, `YYYY-MM-DD`
- Amounts default to 0 if missing
- GST columns are optional (if missing, app estimates from basic/total amounts)
- Duplicate invoice numbers (same as existing invoices) are rejected

---

13. Validation Rules

Invoice Form Validation

Field	Rule	Error Message
Invoice Number	Required, non-empty	"Please fill in required fields"
Party Name	Required, non-empty	"Please fill in required fields"
GST Number	Optional, but if provided must match regex	"GST number must be a valid 15-character GSTIN"
Tax Items	At least one item with amount > 0	"Please add at least one item with a valid amount"
Invoice Number	Must be unique per type (sale/purchase)	"A [type] invoice with this number already exists"

GST Number Format

```
/^[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z]{1}[1-9A-Z]{1}Z[0-9A-Z]{1}$/
```

Breakdown:

```
22  AAAAA  0000  A  1  Z  5
__  _____  ____  _  _  _  _
|   |      |     |  |  |  └─ Check digit
|   |      |     |  |  └──── Always 'Z'
|   |      |     |  └─────── Entity number (1-9 or A-Z)
|   |      |     └──────────── PAN 10th character
|   |      └─────────────────── PAN digits (4 digits)
|   └────────────────────────── PAN letters (5 uppercase)
└────────────────────────────── State code (01-37)
```

Example valid GSTINs: `22AAAAA0000A1Z5`, `07BBBBB1234C2ZD`

CSV Import Validation

Check	Action
File type not `.csv`	Reject with toast error
No data rows	Reject with "must have at least one data row"

Missing DATE	Row marked invalid
Missing INVOICE NO.	Row marked invalid
Missing PARTY	Row marked invalid
Invalid date format	Row marked invalid
Negative amounts	Row marked invalid
Summary/header row detected	Row skipped (not counted as error)
Duplicate invoice number (vs existing)	Import rejected for that record

---

14. Indian GST Explained (For Non-Tax People)

What is GST?

Goods and Services Tax (GST) is India's unified indirect tax system, implemented on July 1, 2017. It replaced multiple state and central taxes with a single tax framework.

Types of GST

Type	Full Name	When Applied
CGST	Central GST	Intra-state (same state) - goes to Central Government
SGST	State GST	Intra-state (same state) - goes to State Government
IGST	Integrated GST	Inter-state (different states) - goes to Central Government

How It Works

Intra-State Transaction (e.g., Delhi seller → Delhi buyer):

```
Item cost: ₹1,000
GST rate: 18%

CGST (9%) = ₹90  → Central Government
SGST (9%) = ₹90  → State Government
Total tax  = ₹180
Total bill = ₹1,180
```

Inter-State Transaction (e.g., Delhi seller → Mumbai buyer):

```
Item cost: ₹1,000
GST rate: 18%

IGST (18%) = ₹180  → Central Government
Total bill  = ₹1,180
```

Tax Rate Slabs

Rate	Common Items
28%	Luxury goods, cars, cement, air conditioners
18%	Most services, electronics, clothing above ₹1,000
12%	Processed food, business class air tickets
5%	Essential goods, transport services
3%	Gold, silver, precious metals
0%	Essential food items (not tracked in this app)

GSTIN (GST Identification Number)

Every registered business gets a 15-character alphanumeric ID:

```
22AAAAA0000A1Z5
│              │
└──────────────┘

  Unique to each business in each state
```

Why Track 15 Columns?

The government requires businesses to report tax collected/paid at each rate separately for CGST, SGST, and IGST. That's:

- 5 rates × 2 (CGST+SGST) = 10 columns
- 5 rates × 1 (IGST) = 5 columns
- **Total = 15 GST columns per invoice**

This app automates this tracking.

---

15. How to Run the Project

Prerequisites

- **Node.js** version 18+ (recommended: 20+)
- **npm** (comes with Node.js)

Installation

```
# Clone or navigate to the project directory
cd /path/to/gst-invoice-manager

# Install dependencies
npm install
```

Development

```
# Start development server with hot-reload
npm run dev

# Opens at http://localhost:3000
```

Production Build

```
# Create optimized production build
npm run build

# Start production server
npm start
```

Available Scripts

Script	Command	Purpose
`dev`	`next dev`	Start development server with hot-reload
`build`	`next build`	Create optimized production build
`start`	`next start`	Serve production build
`lint`	`next lint`	Run ESLint code analysis

Environment

No environment variables or `.env` file required. The app is fully self-contained.

16. UI Component Library

The project uses **Shadcn/ui** - a collection of reusable, accessible components built on top of **Radix UI** primitives and styled with **Tailwind CSS**.

Components Used in the Application

Component	Location	Used By
`Button`	`ui/button.tsx`	Every component (actions, navigation)
`Card`	`ui/card.tsx`	Page sections, form wrapper, summaries
`Input`	`ui/input.tsx`	All form fields, search bar
`Label`	`ui/label.tsx`	Form field labels
`Table`	`ui/table.tsx`	Invoice list, CSV preview
`Select`	`ui/select.tsx`	Tax type dropdown, tax rate dropdown
`Switch`	`ui/switch.tsx`	Inclusive/exclusive tax toggle
`Badge`	`ui/badge.tsx`	Status badges (ACTIVE/CANCELLED/SALE/PURCHASE)
`Toast`	`ui/toast.tsx`	Success/error/info notifications
`Toaster`	`ui/toaster.tsx`	Toast rendering container

Theme System

CSS variables defined in `app/globals.css` :

```
:root {
  --background: 0 0% 100%;      /* White */
  --foreground: 0 0% 3.9%;      /* Near-black */
  --primary: 0 0% 9%;           /* Dark */
  --destructive: 0 84.2% 60.2%; /* Red */
  /* ... etc */
}
```

Dark mode is available via `next-themes` provider but the app primarily uses a light gray background (`bg-gray-50`).

---

17. Known Limitations

Limitation	Impact	Workaround
localStorage only	Data is per-browser, per-device. Clearing browser data deletes everything.	Export CSV regularly as backup
No multi-user	Cannot share data between users or devices	Export CSV and share manually

No authentication	Anyone with browser access can see/modify data	Use on personal/trusted devices
No PDF generation	Cannot create printable invoices	Export to CSV, use Excel for printing
No reports/charts	No monthly/yearly GST summary reports	Export CSV, analyze in Excel
No payment tracking	Cannot track if invoices are paid/unpaid	Manual tracking needed
~5MB localStorage limit	Browser limits localStorage to ~5-10MB	Sufficient for ~10,000+ invoices
No undo	Delete and cancel are permanent	Be careful, cancelled invoices preserved for audit
No GST return filing	Cannot auto-fill GSTR forms	Use exported CSV data for manual filing
No HSN/SAC codes	Does not track item classification codes	Add manually in party/invoice notes

---

18. Possible Future Enhancements

High Priority

- **Cloud Sync**: Firebase/Supabase backend for cross-device access
- **PDF Invoice Generation**: Create printable GST-compliant invoice PDFs
- **Data Backup/Restore**: One-click backup to file, restore from file
- **Monthly GST Summary Report**: GSTR-1 and GSTR-3B style summaries

Medium Priority

- **Payment Status Tracking**: Mark invoices as paid/pending/overdue
- **HSN/SAC Code Support**: Item-level classification for GST returns
- **Party Master**: Reusable party database with saved GSTIN
- **Dashboard Charts**: Recharts-based visualizations (already in dependencies)
- **Keyboard Shortcuts**: Quick-add, save, navigate

Nice to Have

- **Multi-language Support**: Hindi, regional languages
- **Dark Mode Polish**: Full dark mode UI optimization
- **PWA Support**: Offline-first with service worker
- **Mobile-Responsive**: Better mobile form layout
- **Bulk Operations**: Select multiple invoices for export/delete
- **Audit Log**: Track all changes with timestamps

Appendix A: File-by-File Reference

File	Lines	Purpose
`app/page.tsx`	~440	Main application state and layout
`app/layout.tsx`	~20	Root HTML structure and metadata
`app/globals.css`	~80	Tailwind directives and CSS variables
`components/invoice-form.tsx`	~558	Invoice create/edit form with live calculations
`components/invoice-table.tsx`	~260	Searchable, sortable invoice data table
`components/csv-import-frontend.tsx`	~551	CSV upload, validation, preview, and import
`components/frontend-export.tsx`	~132	CSV export (single/separate file modes)
`components/invoice-type-selector.tsx`	~53	Sale vs purchase type picker
`lib/types.ts`	~46	TypeScript interfaces for all data models
`lib/tax-calculations.ts`	~101	GST calculation engine
`lib/local-storage.ts`	~147	localStorage CRUD service with CSV export
`lib/csv-parser.ts`	~84	CSV text parser and date utilities
`lib/utils.ts`	~6	Tailwind class merge utility
`hooks/use-toast.ts`	~190	Toast notification state management
`hooks/use-mobile.tsx`	~19	Mobile viewport detection

Appendix B: Git History Summary

Commit	Message	What Changed
`92ae6c9`	Improve codebase: cleanup dead code, add search/sort/validation	Added search, sort, GST validation, removed unused code
`f488085`	Invoice form changes	Form UI and logic improvements
`a9b37b1`	Update Next.js to fix security vulnerability CVE-2025-66478	Security patch for Next.js
`5e9d024`	Remove Prisma and switch to local storage only	Major architecture change: removed database, went fully client-side
`427f9b2`	Fix build: remove trailing comma and add prisma generate	Build fix
`c7d579e`	Initial commit: GST Invoice Manager	First version of the application

This documentation was generated from the actual source code of the repository. Last updated: March 2026.